Stochastic Gene Expression and Cellular Variability

Joshua S. Weitz[1, 2, *]

[1] *School of Biology, Georgia Institute of Technology, Atlanta, GA, USA*
[2] *School of Physics, Georgia Institute of Technology, Atlanta, GA, USA*

(Dated: September 12, 2020)

I. SIMULATING STOCHASTIC GENE EXPRESSION

Cellular gene regulation arises from the aggregate outcomes of many individual interactions, i.e., between a transcription factor and a promoter, an inducer and a protein, RNA polymerase and a genome sequence, etc. Each of these processes occurs again and again, so that the resulting dynamics are often represented as a type of average, or "mean-field" behavior. The previous chapter introduced computational methods to simulate deterministic dynamics of gene regulation. Those simulation methods are applicable to other scales, whether of tissues, organisms, populations, or ecosystems, The accuracy of deterministic models improve as the number of processes and number of molecules increases. But, one must proceed cautiously, particularly when the numbers are small, which is certainly the case when genes are turned 'On'.

Hence, this laboratory revisits some themes of the previous chapter but with a different goal in mind: simulating stochastic gene expression. Building upon the theory in the textbook, and the previous chapter's development of core programming skills, this laboratory guide will help you understand how to: (i) translate gene expression processes into dynamic events; (ii) translate dynamic events into a simulation; (iii) simulate stochastic gene expression that recapitulates expected mean-field dynamics and that can be compared to measurements. The focal method of this lab is what is commonly known as the Gillespie algorithm. The Gillespie algorithm is a widely used approach developed for simulating chemical reaction kinetics, one event at a time. It can be used in many contexts, including stochastic gene expression. As will be explained below, the core idea of the Gillespie algorithm is that the state of the cell changes based on individual events. Each event is associated with a process, e.g., degradation or production. These processes have rates, such that it is possible to sample the exact time of the next event from an appropriate exponentially

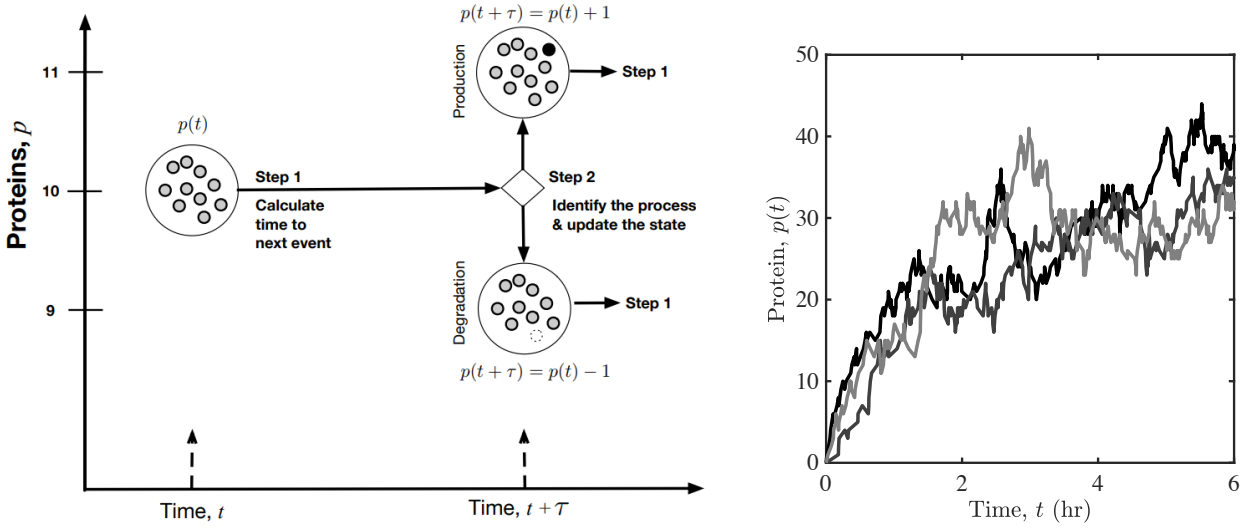


FIG. 1: Gillespie algorithm from schematic to application. (Left) A schematic depicting the discrete change in the number of a state variable, in this case proteins. (Right) The result of implementing the Gillespie algorithm for the simplest model of gene expression.

*Electronic address: jsweitz@gatech.edu; URL: http://ecotheory.biology.gatech.edu

Typeset by REVTEX

distributed random process. Then, at the next event time, the state of the system changes. For example, if a production event occurs, then the number of proteins goes up by one. In contrast, if a degradation event occurs, then the number of proteins goes down by one. Given an updated system, the process repeats and repeats. A schematic of this process is depicted below with multiple realizations of the Gillespie algorithm on the right (see Figure 1).

Here then is the target. Consider a cell in which genes are transcribed and then the mRNA translated into protein. We will aggregate transcription and translation and assume that proteins are produced at a rate β, e.g., 50 or 100 nM/hr. The units are important. For example, a single protein in the volume of an *E. coli* cell is approximately 1 nM concentration. Because cells are growing, we will assume that proteins decay at an exponential rate of 1 hr^{-1}. This oversimplifies the actual change in concentrations due to cell growth, division, and active degradation but will suffice, for now. This lab will explore the consequences of tracking the dynamics of individual proteins rather than their concentrations. Each section is organized around a series of questions. Will proteins decay away to nearly zero, increase without bound, or will they reach a steady state? How much variation do we expect to find between cells or when examining a cell at different times? In addition, given that stochasticity is now embedded into the core dynamics, we will need to revisit the very notion of a 'steady state'.

The lab focuses on the one-dimensional case, i.e., where the state space is represented by the numbers of proteins. However, the Gillespie algorithm is more general, and can also be used to simulate the stochastic dynamics of multi-component systems, e.g., those involving mRNA and proteins, or as part of feedback systems with mRNA, proteins, inducers, etc., and eventually as part of entire gene regulatory networks. A secondary goal of this lab is to provide a gateway for simulating seemingly complex models that might not be so complex to simulate after all. The textbook examines the evidence associated with transcriptional bursting, a topic explored at depth in the homework, made possible by the insights and methods provided in the sections to come.

II. POISSON PROCESSES: FINDING THE TIME OF THE NEXT EVENT

A. Waiting time theory

How long does it take for the next production event to take place if the *rate* is b? This rate has units of inverse time, e.g., hrs^{-1}. There tends to be significant misunderstanding at what such units means. One way to think about them is that the inverse of the rate is a characteristic time, approximately equal to that of the time between random events. So events with a rate of 4 hrs^{-1} occur about every 15 minutes, whereas events with a rate of 10 hrs^{-1} occur about every 6 minutes. *Put simply: higher rates imply shorter intervals between events.*

Formally, this rate should be thought of as a *probability per unit time* that an event occurs. The probability that an event occurs in some small interval dt is $b \times \text{dt}$. The probability that an event does not occur is $1 - b \times dt$. For some, you might recognize such events as arising from a Poisson process. With rates defined, the question we want to ask is: *what is the probability that the event takes place at some time t from now.* For example, if an event occurs at a rate of 5 per hour, then what is the probability that the *next event* occurs precisely 0.2 hours, 0.4 hours, or 2 hours in the future?

For an event to occur at time t for the first time, it should not occur anytime before-hand. This seems obvious, but it's the key to moving from rates to probabilities. Recall that $p(t)\text{dt}$ is the probability that the event occurs between time t and time $t + \text{dt}$ where dt is some small time interval, e.g., 0.00001 hrs. Therefore, for the event to occur at time t, then it should satisfy the following equation:

$$p(t)\text{dt} = \overbrace{(1 - b\text{dt}) \ldots (1 - b\text{dt})}^{\text{Did not occur } \frac{\text{t}}{\text{dt}} \text{ times}} \times \overbrace{b\text{dt}}^{\text{did occur}} \tag{1}$$

In words, this means that the event does not occur in any of the $\frac{t}{\text{dt}}$ intervals between 0 and t. Then, the event does occur with probability $b$dt in the time interval between t and $t + \text{dt}$.

Next, keep in mind that $b\text{dt} \ll 1$, i.e., the probability is very small given small time intervals. Hence, we can approximate $1 - b\text{dt} \approx \text{e}^{-b\text{dt}}$. Combining this together we find

$$\begin{aligned} p(t)\text{dt} &= e^{-b\text{dt}\frac{\text{t}}{\text{dt}}} \cdot b\text{dt} \\ p(t)\text{dt} &= e^{-bt} b\text{dt} \end{aligned}$$

This is the key result. A process that occurs at a rate b leads to *exponentially* distributed events. This is termed a *Poisson* process, i.e., one in which the average time between events is fixed, but the exact time of the next event is unknown and sampled from a memory-less (exponential) distribution.

B. Some computing

There are multiple ways to generate exponentially distributed random numbers. As explained in the previous chapter, the straightforward approach is to use the built-in random number generator in Python:

```
import numpy as np
import matplotlib.pyplot as plt
b=3
np.random.exponential(1/b)
```

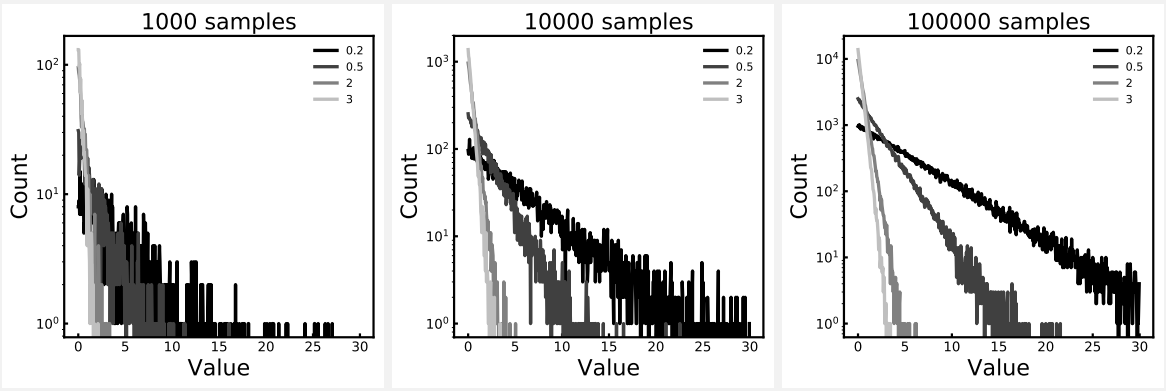
Here, the rate is 3 per hr, and the function `np.random.exponential` takes a single input – the inverse of the rate – and returns an exponentially distributed random number. Hence, the input argument to `np.random.exponential` should be the expected mean of the exponentially distributed set of random numbers. Verify that this works by generating 10,000 such random numbers, following the code snippet below to find their mean:

```
tvals = np.random.expoential(1/b,10000)
meant = np.mean(tvals)
```

As you notice, the mean you measure not *exactly* what you expected given the input rate parameter – 1/3? This is expected, given that these are chosen randomly, but the mean is indeed very close to 1/3 and will be ever closer the more samples you take (at least on average). Next up: a challenge problem.

Challenge Problem: Exponentially distributed random numbers

Plot the probability distribution for 10^4 and 10^5 exponentially sampled random numbers. Are the realized distributions in fact exponential? Now, change the rate parameter, e.g., setting it to $b = 0.2$, 0.5, 2 or 3. Recall that the `hist` command is one way to visualize distributions, although there are other options as well. If your code works, the result should look something like the following:



Solutions to Challenge Problem: Exponentially distributed random numbers
The following code generates samples of 10^3, 10^4 and 10^5 random numbers (by varying the variable `num`). One could plot the cumulative distribution function (cdf) or the probability distribution function (pdf). This code uses the pdf and it is apparent by plotting the histograms on logarithmically scaled y-axes that the distributions are in fact exponential. The slopes of the straight lines correspond to the values of b, denoted in the legend. Note that the estimates of the rate parameter, correspond closely but not exactly to the generating value. However, the 95% CI-s are almost certainly going to include the mean – see the text for details.

```
import numpy as np
import matplotlib.pyplot as plt

# Setup
num = 1000
bvals = [0.2, 0.5, 2, 3]
b_est = np.zeros(np.shape(bvals))
tmpcol = np.array([[0, 0, 0],
                   [0.25, 0.25, 0.25],
                   [0.50, 0.50, 0.50],
                   [0.75, 0.75, 0.75]])

# Loop for each conditions of b
for i in range(len(bvals)):
    #generate plots
    b=bvals[i]
    #sample at random
    x=np.random.exponential(1/b,num)
    #take a histogram with uniform bins
    figTemp = plt.figure(1)
    bins = np.arange(0,30.05,0.05)
    [tmpcount,tmpx,patches]=plt.hist(x,bins)
    figReal = plt.figure(2,figsize=(5,5))
    ax = figReal.gca()
    plt.semilogy(tmpx[:-1],tmpcount,color=tmpcol[i,:],linewidth=3)

# Plot labeling
plt.xlabel('Value',fontsize=20)
plt.ylabel('Count',fontsize=20)
plt.title('{num} samples'.format(num=num),fontsize=20)
plt.legend(list(map(str, bvals)),frameon=False)

plt.setp(ax.spines.values(),linewidth=2)
ax.tick_params(labelsize=12,direction='in',width=2)
```

It is also possible to calculate the confidence interval in the unknown value of b by fitting an exponential to the measured values in `x`. These confidence intervals (CIs) are a means to quantify the extent to which an unknown value of $\hat{b}$ could have plausibly generated the observed set of random numbers. It is worth modifying the code snippet to include much smaller sets of random samples – in which case the confidence interval will expand. The code snippet to generate these CI-s is as follows:

```
# Number of samples
num = 10**5

bvals = [0.2, 0.5, 2, 3]
b_est = np.zeros(len(bvals))
b_ci = np.zeros((len(bvals),2))
# Loop for each condition of 'b'
for i in range(len(bvals)):
    b = bvals[i]
    x = np.random.exponential(1/b,num)

    # Best estimates for the mean
    b_est[i] = 1/np.mean(x)
    #Python does not have a built-in way to calculate confidence intervals
    # use bootstrapping
    tmpn=1000
    bootstraps_est = np.zeros(tmpn)
```

```
    for j in range(tmpn):
        #Sample from x, allow repeats
        tmpind = np.random.randint(num, size=num)
        bootstrap_x=x[tmpind]
        bootstraps_est[j] = 1/np.mean(bootstrap_x)
    bootstraps_sorted = np.sort(bootstraps_est)
    b_ci[i,0] = bootstraps_sorted[int(tmpn*0.025)]
    b_ci[i,1] = bootstraps_sorted[int(tmpn*0.975)]
```

By modifying the value of $\hat{b} = 0.2$, 0.5, 2, and 3, the analysis yields:

b	$n = 10^4, \hat{b}$	CI	$n = 10^5, \hat{b}$	CI
0.2	0.203	$[0.199, 0.207]$	0.199	$[0.198, 0.200]$
0.5	0.503	$[0.493, 0.513]$	0.501	$[0.500, 0.504]$
2	2.034	$[1.99, 2.07]$	2.00	$[1.99, 2.01]$
3	2.984	$[2.93, 3.04]$	2.99	$[2.97, 3.01]$

III. A THEORY OF TIMING GIVEN MULTIPLE, STOCHASTIC PROCESSES

The prior section delineates the relationship between the rate of a Poisson process and the timing of the next event. But, what happens if more than one kind of event can take place independently, each with their own characteristic rate? What then is the expected time before any event of either kind takes place? This next section addresses the issue of identifying the time before *either* process 1 occurs OR process 2 occurs. For example, if proteins can be produced or degraded, then seemingly some event will happen faster than when only consider production alone or degradation alone. Note that consistent with the usage thus far, we will assume units of hrs for time and hrs^{-1} for rates.

To begin, consider two independent Poisson processes with rates $b_1 = 0.5$ and $b_2 = 2.5$. Given those two rates, what is the expected time until the next event? This question can be answered computationally as a guide to the intuition developed in the main text. One way to answer this equation is to sample two exponentially distributed random numbers using `np.random.exponential` (or using the conversion formula leveraging `np.random.uniform` above). Then, the waiting time should correspond to the *smaller* of these two waiting times, as in:

```
# Set the rates
b1=0.5
b2=2.5
# Generating the anticipated event times
r1=np.random.exponential(1/b1)
r2=np.random.exponential(1/b2)
# Find the smaller time
etime1 = min(r1,r2)
```

The smaller time stored in `etime1` corresponds to the event that occurs first. The function `min` selects the minimum value amongst the two exponentially distributed random numbers. As an intuitive check, run the code and then ask yourself: was the number closer to 2 (the inverse of `b1`) or closer to 0.4 (the inverse of `b2`)? In reality, a single event is insufficient to help build up one's intuition. Instead, repeat the above procedure 1000 times, recording the times in an array. Here is one way to do it:

```
etime = np.zeros(1000)
for i in range(1000):
    b1=0.5
    b2=2.5
    r1=np.random.exponential(1/b1)
    r2=np.random.exponential(1/b2)
    etime[i] = min(r1,r2)
```

Note that it is possible to write a version of this code with `if` statements – try to figure this out on your own.

Irrespective of how you calculated it, consider the following two questions:

- What is the distribution of the variable `etime`, in other words the expected time to the first event?
- What is the average time to the first event?

Whether plotting the pdf using the `hist` command, or using the cdf, you will notice that the events appear exponentially distributed. They are! The reason is that rates of events add. In other words, the next event occurs as if there was a single Poisson process with rate $b = b_1 + b_2 = 3$ per hour! This means that the average time is closer to 1/3 of an hour (i.e., faster than either process alone!).

Formally, we can compare the distribution to the expected distribution from a Poisson process with $\lambda = 3$. When comparing continuous distributions it is often useful to compare the cumulative distributions. If the data of times between events is saved as a variable `etime`, then the following code snippet compares the realized cdf vs. the theoretically expected cumulative distribution function. Define $Q(t)$ as the cumulative distribution function, i.e., the probability that the event happens at any time between $0 \leq t' \leq t$, then

$$Q(t) = \int_0^t \mathrm{dt'p(t')} = \int_0^{\mathrm{t}} \mathrm{dt'be^{-bt'}}, \tag{2}$$

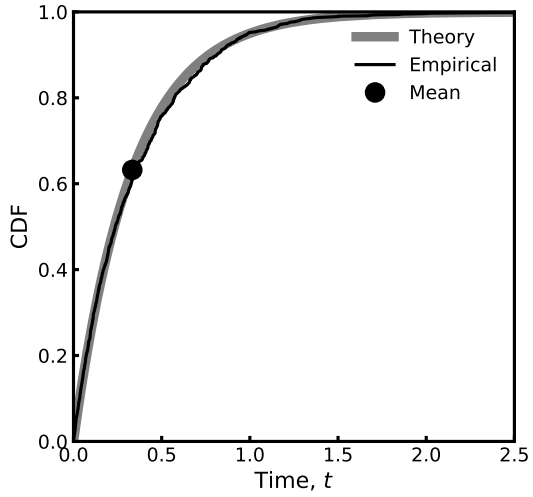
$$= -e^{-bt'}\Big|_0^t, \tag{3}$$

$$= 1 - e^{-bt}. \tag{4}$$

The empirical cdf can be calculated by sorting the n sampled values, and finding the fraction of the values whose timing is less than that, for every point $1 \ldots n$. The following code compares the empirical cdf to the theoretical expectation. Note that this first version of the code does not include comments. Try to figure it out yourself, and then look at the documented version following the figure to unpack the meaning of each section.

```
fig = plt.figure(figsize=(5,5))
ax = fig.gca()
b1 = 0.5
b2 = 2.5
n = 1000
x1 = np.random.exponential(1/b1,n)
x2 = np.random.exponential(1/b2,n)
etime = np.min(np.array([x1,x2]),axis=0)
sortdata = np.sort(etime)
cdfvals = np.arange(n)/n
poiss3cdf = lambda t: 1-np.exp(-(b1+b2)*t)
plt.plot(sortdata,poiss3cdf(sortdata),color=[0.5,0.5,0.5],linewidth=6)
plt.plot(sortdata,cdfvals,color='k',linewidth=2)
plt.plot(1/(b1+b2),poiss3cdf(1/(b1+b2)),marker='o',color='k',
         markerfacecolor='k',markersize=12,linestyle='')
plt.legend(['Theory','Empirical','Mean'],fontsize=12,frameon=False,
           loc='upper right')
plt.xlim(0,2.5)
plt.ylim(0,1)
plt.xlabel(r'Time, $t$',fontsize=14)
plt.ylabel('CDF',fontsize=14)
plt.setp(ax.spines.values(),linewidth=2)
ax.tick_params(labelsize=12,direction='in',width=2)
```

The result of this code should lead to the following:



This code has done a few things. Let's diagnose it, including comments for added explanation.

```
#Initialize figure
fig = plt.figure(figsize=(5,5))
ax = fig.gca()

#Generate 1000 random numbers based on the minimum of two processes
b1 = 0.5
b2 = 2.5
n = 1000
x1 = np.random.exponential(1/b1,n)
x2 = np.random.exponential(1/b2,n)
etime = np.min(np.array([x1,x2]),axis=0)

# Sorts the event times from smallest to largest
sortdata = np.sort(etime)

# Determines the fraction of the data less than a particular value
# resulting in the cumulative distribution
cdfvals = np.arange(n)/n

# Calculate the cdf of the predicted exponential distribution
poiss3cdf = lambda t: 1-np.exp(-(b1+b2)*t)

#Plot the predicted CDF as a thick gray bar
plt.plot(sortdata,poiss3cdf(sortdata),color=[0.5,0.5,0.5],linewidth=6)
# Plot the empirical diistribution
plt.plot(sortdata,cdfvals,color='k',linewidth=2)
# Overlays the location of the expected mean value
plt.plot(1/(b1+b2),poiss3cdf(1/(b1+b2)),marker='o',color='k',
         markerfacecolor='k',markersize=12,linestyle='')

#Add a legend, set axis limits and labels and increase axis thickness and
# font for readability
plt.legend(['Theory','Empirical','Mean'],fontsize=12,frameon=False,
           loc='upper right')
plt.xlim(0,2.5)
plt.ylim(0,1)
plt.xlabel(r'Time, $t$',fontsize=14)
plt.ylabel('CDF',fontsize=14)
plt.setp(ax.spines.values(),linewidth=2)
ax.tick_params(labelsize=12,direction='in',width=2)
```

With this code complete, it is time for the next challenge.

Challenge problem: Identity of Random Events

This challenge is centered on solving the following: given two independent Poisson processes operating concurrently, what fraction of times do each of the two processes take place first? That is to say, if $b_1 = 0.5$ and $b_2 = 2.5$, what fraction of events are associated with process 1 and what fraction of events are associated with process 2? Write your own code to try and solve this, leveraging the code already written thus far in the laboratory guide.

Solutions to Challenge Problem: Identity of random events.
The theoretical expectation is that a Poisson event that occurs at a rate b_i amongst S events occurs before all others a fraction $\frac{b_i}{\sum_i b_i}$ of the time. Hence, in the case where $b_1 = 0.5$ and $b_2 = 2.5$, then the slower process 1 will occur 0.5/3 or 1/6-th of the time and the faster process 2 will occur 2.5/3 or 5/6-th of the time. The following code snippet shows how to do this, even if you did not know or are not convinced by this argument. The lecture notes provide a detailed derivation of why this claim holds in general.

```
# Generate random numbers
num=10**3
b1=0.5
b2=2.5
b1_rnd=np.random.exponential(1/b1,num)
b2_rnd=np.random.exponential(1/b2,num)

# Find out which came first
tmpi1=np.argwhere(b1_rnd<b2_rnd)
tmpi2=np.argwhere(b2_rnd<=b1_rnd)
print('Fraction of 1-events that come first is',len(tmpi1)/num)
print('Fraction of 2-events that come first is',len(tmpi2)/num)
```

This section, and challenge problem, suggests an alternative method of simulating. Instead of choosing the shortest time, we could first independently sample the time according to a Poisson process with a rate corresponding to any process occurring. In general, the rate of any process occurring is $\lambda = \sum_j \lambda_j$, where λ_j is the rate of a single process, j and the sum includes all subprocesses. Next, we would sample which reaction occurs. The subprocess, j, is chosen with probability $\frac{\lambda_j}{\sum_j \lambda_j}$, where the sum includes all subprocesses. This approach is at the core of the Gillespie algorithm.

IV. GILLESPIE ALGORITHM APPLIED TO A GENE EXPRESSION MODEL

A. Conceptual framework

The previous section demonstrated how to identify the waiting times between events given two Poisson processes with constant rates. But rates can also depend on the state of the system. For example, in a gene regulatory system, many reactions occur at rates proportional to the concentration of molecular types, e.g.,:

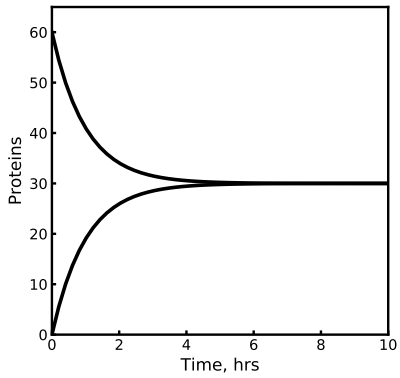
- Substrate and enzyme
- Transcription factor and DNA binding site
- Receptor and ligand

There is another difference between the next challenge and the prior work. When an event occurs inside the cell, then the degradation, production, or transformation of a molecule changes the state of the system and therefore has the potential to alter the underlying rates of the governing processes. This feedback is at the core of the Gillespie algorithm.

Returning to the motivating example, consider a cell in which proteins are produced at a constant rate β and decay at a rate α. The abundance of proteins, p, can be represented in terms of a differential equation:

$$\frac{dp}{dt} = \overbrace{\beta}^{\text{production}} - \overbrace{\alpha p}^{\text{decay}}. \tag{5}$$

Solving nonlinear equations of gene regulatory dynamics can't necessarily be solved analytically, though this particular choice of system does have an accessible closed form solution. However, it is possible to simulate dynamics arising from such equations via numerical integration. If one numerically integrates this equation given $\beta = 30$ nM/hr^{-1} and $\alpha = 1$hr^{-1}, the system converges to a fixed amount. Even without integrating, it should be apparent that there is a protein abundance at which production balances decay. That balance happens when $p^* = \beta/\alpha$.



Here, the two lines depict the dynamics of protein concentration given a continuous model, starting from two different initial concentrations, 0 and 60 proteins, and an equilibrium of 30.

But, are these continuous curves realistic? What happens in a real cell where there can be 0, 1, 2, ..., 29, 30, 31, ... of proteins but not 29.231312 proteins? How can a stochastic algorithm capture the random nature of protein production? According to the Gillespie algorithm, the total rate at which any event occurs is the sum of the rates of individual processes:

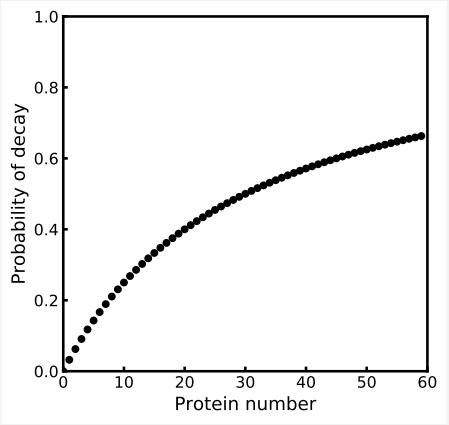
Production Occurs at a constant rate β, independent of protein concentrations.

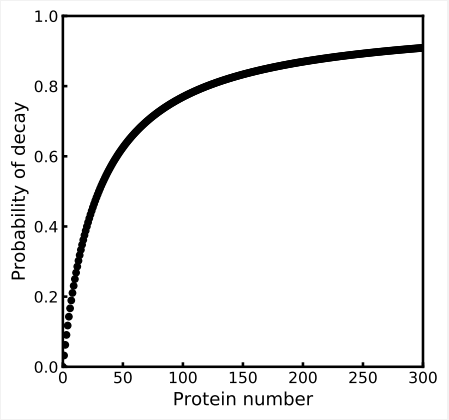
Decay Occurs at a protein-dependent rate αp.

Based on these two processes, first try to sketch the two rates as a function of the protein concentration. Ask yourself: how do they compare and is there a critical protein concentration where the two rates cross (and what exactly might that mean)?

Challenge Problem: State-Dependent Rates

Write code that calculates the probability of decay as a function of the state p. Using this code, calculate the probability of decay at equilibrium, i.e., when $p^* = \frac{\beta}{\alpha} = \frac{30}{1}$. Use code and visualizations to support your argument. If your code is working, it should look like this, depending on whether you visualize from 0 to 60 proteins (left) or from 0 to 300 (right):





Solutions to Challenge Problem: State-dependent rates.
The probability of a decay event taking place before a production event is the ratio of the decay rate to the total rate of all processes. Hence, it is $\alpha p/(\beta + \alpha p)$. That is a saturating function of p. Note that at equilibrium then $\beta = \alpha p^*$, such that by definition, the probability of decay is 1/2, the probability of production is 1/2, and so the system is equally likely to increase protein levels by 1 as it is to decrease protein levels by 1. The visualization of the probability of decay is enabled by the following code:

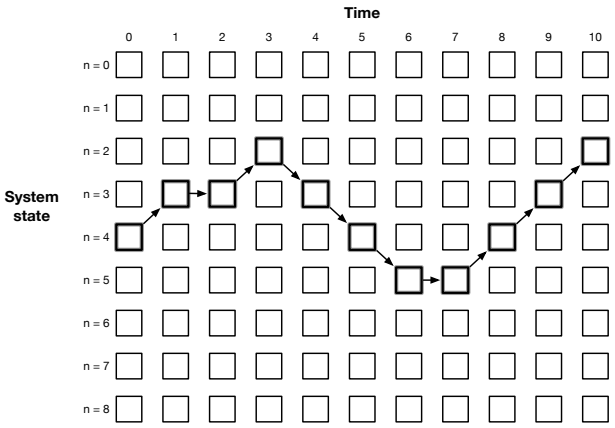
```
# Set rates and probability of decay
alpha = 1
r = 30
probdecay = lambda p: alpha*p/(r+alpha*p)
# Plot the probability of decay as a function of proteins
pvec = np.arange(300)
plt.plot(pvec,probdecay(pvec),color='k',marker='.',markersize=10,linestyle='')
plt.ylim(0,1)
```

B. Gillespie algorithm

Before implementing the full algorithm, it is worth discussing the algorithm in a generic form. Computer scientists often call this "pseudocode". Pseudocode is not written in a specific language but nonetheless lays out a series of steps that your actual code might follow. Here is a pseudocode version of the Gillespie algorithm:

```
specify time range of simulation
set initial time vector and protein numbers vector to 0
while the current time is less than the maximum
        update rate of decay based on number of proteins
        update rate of production
        find the waiting time until the next event
        update the current time based on the waiting time
        decide on which process occurred
        update and store the number of proteins based on selected event
        store the information about the system state at the current time
```

This pseudocode represents an ideal time to pause, strategize, and ideally discuss each step with a classmate or study partner. Try to draw a sketch of what might happen given a cell that has initially 4 proteins vs. one in which there are initially 1 protein. Is every trajectory the same? Note that in this case, the rate of production remains constant irrespective of the current protein level. This need not be the case generally, e.g., as part of gene regulatory networks in which the current expression levels influence new production. In doing so, the following schematic may be helpful. In reflecting on its meaning, also consider: what would happen if the system started with 0 proteins, could the gene still turn 'On'?



C. Implementing stochastic gene expression

It is time to build a model of stochastic gene expression using the Gillespie algorithm. The following is one version, feel free to follow it or implement a version on your own! Start a script, and enter the following (though the comments are optional):

```
#stochastic gene expression
#part 1 - initial conditions
p0=0 #initial proteins
alpha = 1 #decay rate
beta = 30 #production rate
currp = p0 #current proteins
tmax = 6 #max time
currt = 0 #current time
tvals = [currt] #recording of time
pvals = [currp] #recording of proteins
#end of part 1 - initial conditions
```

With this in hand, now start simulating:

```
# Part 2 - Stochastic simulation
while currt<tmax:
```

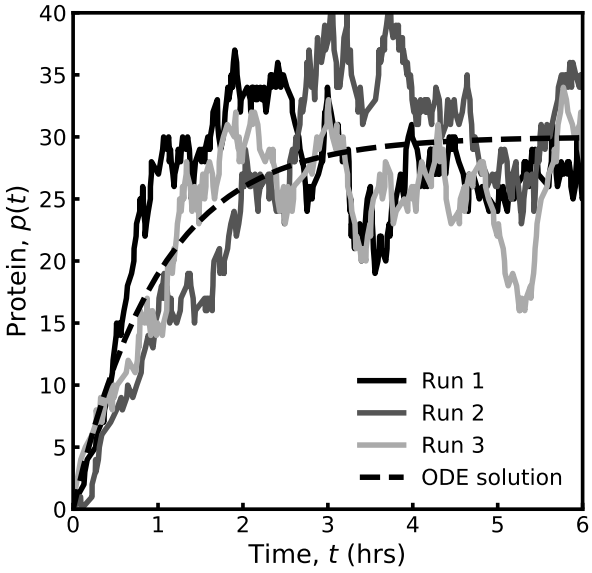
```
# Calculate rates
decayrate = alpha*currp
prodrate = beta
total_rate = decayrate+prodrate

# Find waiting time
deltat = np.random.exponential(1/total_rate) # Find next event
currt = currt+deltat # Move forward in time
prob_produce = prodrate/total_rate # Calculate probability of production

# Update the state
if (np.random.uniform()<prob_produce): # Choose production at random
    currp = currp+1 # Increment number of proteins by 1
else:
    currp = currp-1 # Decrement number of proteins by 1

# Record the state
tvals.append(currt)  #Store the event time
pvals.append(currp) #Store the protein number
```

That's it! Now, plot a stochastic trajectory using the values stored in `pvals`. Three trajectories are plotted below to show some typical variation in the dynamics. The protein number approaches the expected state value of $p^* = \frac{\beta}{\alpha} = \frac{30}{1}$ and then appears to fluctuate about this value. The inherent noise from the sampling process causes the fluctuations around what would be an otherwise stable equilibrium for the analogous ODE dynamics. Here, the plot also includes the expected dynamics from the ODE model - it is not a coincidence that the ODE solution runs through what appears to be the center of the stochastic model simulations. As explained in the main text the ODE solutionsolution is the expected mean field (averaged) behavior of the stochastic trajectories.



Using this code, try out a few variations:

- Extend or shorten the maximum time
- Double the production, leaving the decay the same
- Double both the production and decay rates – does the mean change? If not, do you see any difference?
- Explore the system on your own ...

D. Bonus: discrete sampling of stochastic trajectories

Comparing stochastic trajectories across time can be difficult because the events occur at random times. It can be useful to sample the trajectory at fixed times. Sampling at fixed times would seem to be intrinsic to the nature of experimental design, but an event-driven simulation reports back changes in values at exponentially distributed intervals – these are almost certainly

not equally spaced! Hence, the next objective is to learn how to write a function to sample a stochastic trajectory at fixed times. This pseudocode may be helpful:

```
Choose vector of times.
pre-allocate a vector the same size as the vector of times for sampled protein numbers
loop over vector of times
find the last protein number at the time less than or equal to the current time
place this value into the vector of sampled protein numbers
end
```

This simple concept is powerful, see for example Figure 2, in which the results of a single stochastic trajectory is contrasted with the sampled version at a frequency of 10x and 2x per hr. Such sampling also makes it possible to compare one trajectory to the next. The following figure shows interval sampling of stochastic trajectories, comparing the process (top, circles) with sampled output at 10x per hr (middle, circle and black lines) and sample output at 2x per hr (bottom, circle and lines).

Challenge Problem: Discrete sampling of stochastic trajectories

Implement a function to take the output of an event-driven Gillespie algorithm and return the values of the state at discrete or otherwise pre-specified intervals. This will be useful in moving from the system perspective to the measurement perspective – see Figure 2.

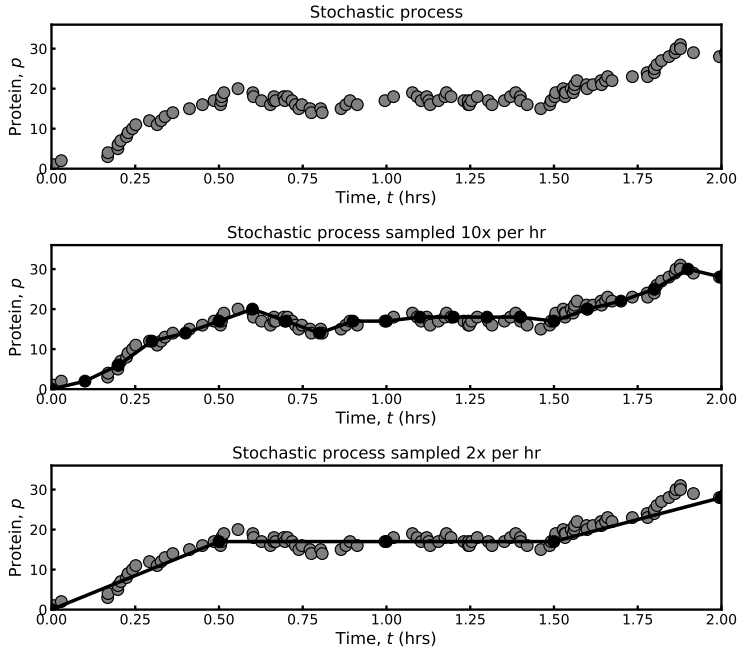


FIG. 2: Sampling the stochastic trajectory at lower frequencies may lose some of the details of the trajectory.

Solutions to Challenge Problem: Discrete sampling of stochastic trajectories. There are many ways to accomplish this goal. Below is a function which implements the pseudocode described in the laboratory text. The function takes the observed time and state values, `t` and `y`, respectively, as input along with a specific range of times `trange`. The function then returns the times and state values. In essence, the function moves from one sample time point to the next and then scans through the event times until the event time exceeds the sample time. At that point, the value of the state is saved, and then the loop advances. This function retrospectively samples times and states, irrespective of the event timings returned by the Gillespie algorithm. Note that a full code would also deal with the possibility of boundary checking, i.e., ensuring that the sample times and event times are bounded.

```
def sample_traj(t,y,trange):
    """Samples a trajectory t,y at discrete intervals"""
    t = np.array(t)
    y = np.array(y)
    ts = np.zeros(np.shape(trange))
    ys = np.zeros(np.shape(trange))
    #initialize
    curt = trange[0]
    ind=np.argwhere(t<=curt)
    curind = ind[-1]
    ts[0]=t[curind]
    ys[0]=y[curind]

    #Scans across the sample interval and then moves the
    # actual dynamics forward until we cross it
    for i in range(1,len(trange)):
        nextt=trange[i]
        while t[curind]<nextt:
            curind=curind+1
        ts[i]=nextt
        ys[i]=y[curind-1]
    return ts, ys
```

With this code in place, a comparison can be made as follows

```
# Run the Gillespie algorithm
alpha=1
beta=30
tmax=2

# Plot the original
t,y = stochsim_protein([0, tmax],0,beta,alpha)
tmph=plt.plot(t,y,'ko','grey')

# Sample the trajectory every 0.1 hrs
ts,ys=sample_traj(t,y,np.linsapce(0,tmax,0.1))

# Plot the sampled trajectory
tmph=plt.plot(ts,ys,'ko-',linewidth=3)
```

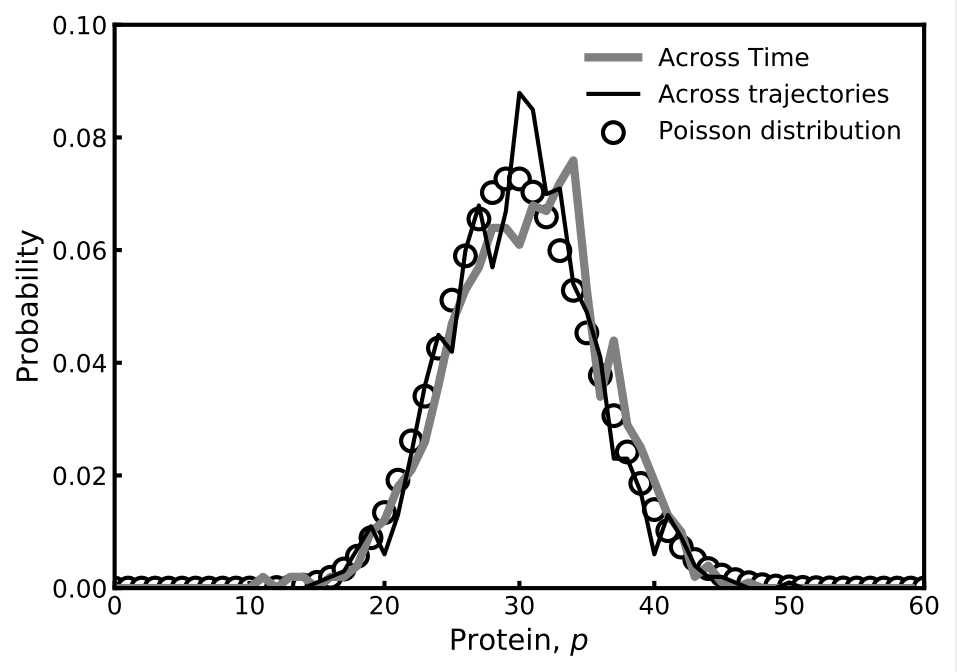
E. Double bonus: comparing statistics of the 'steady state'

The double bonus – appearing for the first time in this laboratory. If you have made it this far, you now have a working Gillespie algorithm and a means to sample these stochastic trajectories at fixed intervals. Now you are ready for the next challenge: to compare statistics of the stochastic protein values across time and across distinct trajectories altogether. To do so, first tun the Gillespie algorithm with initially 0 proteins for a long time–say 20 hours. Use the last point as the initial condition for a new simulation. Run the simulation for another 100 hours. Sample this trajectory between 20 and 120 hours for 1000 equally spaced time points (this corresponds to sampling every 6 minutes). What is the mean and variance of the protein numbers across time for this simulation? What is the distribution of protein numbers across time? Now repeatedly simulate the dynamics for 20 hours starting with 0 proteins each time. Do this for 1000 runs. For each simulation store the final protein number. What is the mean and variance of the final protein numbers across the trajectories? Compare this distribution of final protein numbers to the distribution across time that we computed above. The results are remarkably... the same! The

reasons why are deep, but reflect the 'ergodicity' of the problem, the fact that a sufficiently spaced sample of the system's steady state is representative of the entire state space. The following challenge problem formalizes this, though given time constraints you might want to only take on one of the two ways of calculating the steady state. The other remarkable thing is that the distribution itself is a Poisson distribution – for reasons why see the main text.

Challenge Problem: Steady State Distributions of Stochastic Gene Expression

Develop code that compares and contrasts the output of the long-term sampling of a single trajectory vs. sampling of many trajectories at distinct points. Use $\beta = 30$, $\alpha = 1$, and sample at the 20 hr point across 1000 runs or sample every 0.1 hrs after reaching the 20 hr point. If your code works it should look like:



Notice, the distributions are nearly the same. The notion that the state fluctuates across time in the same manner that it fluctuates across replicates is referred to as ergodicity. The main take-away of ergodicity is that if the dynamics are not expected to change significantly an experimentalist can choose to obtain statistics by either repeating an experiment or by increasing the number of samples of a trajectory. This is not expected to hold when transients are a major factor; hence, we simulated 20 hours after the initial condition before taking data.

Solutions to Challenge Problem: Steady State Distributions of Stochastic Gene Expression. The solution to this challenge problem involves running stochastic trajectories, sampling, saving, and repeating. The following code snippet provides sufficient detail to reproduce the main results:

```
# Parameters
beta = 30
alpha = 1

# Long term trajectory
t, y = stochsim_protein([0,20],0,beta,alpha)
y0 = y[-1]
t, y = stochsim_protein([0,100],y0,beta,alpha)
trange = np.linspace(0,100,1001)
ts, ys = sample_traj(t,y,trange)

# Histogram
py,px = np.histogram(ys,np.arange(0,61))
plt.plot(px[:-1],py/sum(py),linewidth=4,color=[0.5,0.5,0.5])

# Simulate many trajectories
numsamps = 1001
final_y0 = np.zeros(numsamps)
for j in range(numsamps):
    t, y  = stochsim_protein([0,20],0,beta,alpha)
    final_y0[j] = y[-1]

py2,px2 = np.histogram(final_y0,np.arange(0,61))
plt.plot(px2[:-1],py2/sum(py2),linewidth=2,color='k')

# Calculate the distribution in theory
px_theory = np.arange(0,61)
py_theory = stats.poisson.pmf(px_theory,beta/alpha)
plt.scatter(px_theory,py_theory,c='w',edgecolors='k',s=100,linewidth=2)

#Additional plot labels at your discretion
```

V. LOADING AND SAVING DATA

Loading and saving `.csv` files is easy in Python. First, write your trajectory data as a row in the `.csv` file by changing the following strings.

```
import csv
myFile = open('myfilename.csv','w')
with myFile:
    writer = csv.writer(myFile)
    writer.writerow(myarrayname)
```

Then reload it using numpy's built-in reader.

```
x=np.genfromtxt('myfilename.csv',delimiter=',')
```

If the `.csv` file has a header, the `skip_header` argument may denote the number of rows to skip.

VI. TAKE-AWAY POINTS

The major take-away points from this lab include

- Random events that occur at each moment with the same rate independent of previous events are term Poisson processes.
- The time between events in a Poisson process are distributed exponentially.
- The average time between events in a Poisson process is equal to the inverse of the rate.

- When multiple events take place, the average time to the next event (of any kind) is equal to the inverse of the *sum* of the underlying rates.
- The Gillespie algorithm represents event-driven dynamics, where the time between events is exponentially distributed, then the state is updated given the event type, and the process repeats
- A stochastic simulation of protein dynamics recapitulates the deterministic model, on average.
- The steady state of a stochastic simulation of protein dynamics is a random state variable whose value is distributed like a Poisson distribution.
- The Gillespie algorithm can be extended to multi-dimensional state spaces and many types of processes using the same core techniques.